DESIGN AND SIMULATION OF A 4-BIT RIPPLE CARRY ADDER USING LOGISIM EVOLUTION

Internship Project Report: Project No. 2

Student Name	Sahitya
Programme	B.Tech, Electronics and Communication Engineering (ECE)
Year	Third Year
Project Type	Internship Project
Academic Year	2026–2027
Institution	ANU COLLAGE
Internship Organization	Codtech Intern
Date of Submission	20-10-2026

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Software and Tools Required
5. Theoretical Background
6. Half Adder Design
7. Full Adder Design
8. 4-Bit Ripple Carry Adder Architecture
9. Circuit Design in Logisim Evolution
10. Truth Tables
11. Testing and Verification
12. Circuit Diagrams and Screenshots
13. Results and Discussion
14. Applications
15. Advantages and Limitations
16. Learning Outcomes
17. Conclusion
18. Future Scope
19. References
20. Viva Voce Questions and Answers

1. Abstract

Arithmetic operations are at the heart of every digital system, and addition is the most fundamental of them. This project presents the design and simulation of a 4-bit Ripple Carry Adder (RCA) using Logisim Evolution, a free graphical tool for designing and simulating digital logic circuits. The circuit accepts two 4-bit binary numbers, A (A3 A2 A1 A0) and B (B3 B2 B1 B0), and produces a 4-bit sum (S3 S2 S1 S0) together with a final carry output (Cout).

A Ripple Carry Adder is built by connecting several Full Adders in cascade. Each Full Adder adds one bit of A, one bit of B and a carry coming from the previous stage, and produces a sum bit and a carry bit. The carry output of each stage is connected to the carry input of the next stage, so the carry “ripples” from the least significant bit to the most significant bit. This gives the adder its name.

In this project, a Half Adder was first built to understand single-bit addition. A Full Adder was then constructed from XOR, AND and OR gates and packaged as a reusable subcircuit named FullAdder. Four instances of this subcircuit were connected in a chain, with the first carry input fixed at logic 0, to form the 4-bit adder. Logisim Evolution was used to apply different input combinations and observe the outputs. This report describes the theory, design steps, test cases and applications of the circuit, and it clearly separates expected results from results verified in the simulation.

2. Introduction

2.1 Digital Electronics

Digital electronics deals with circuits that process information represented by two discrete voltage levels. These levels are interpreted as logic 0 (low) and logic 1 (high). Because only two levels are used, digital circuits are far less sensitive to noise than analog circuits and are easy to design, store, replicate and scale. Computers, mobile phones, communication equipment, calculators and embedded controllers are all built from digital circuits.

2.2 Binary Addition

In the decimal system, two numbers are added digit by digit, starting from the rightmost digit, and a carry is passed to the next column whenever a column total exceeds 9. Binary addition follows exactly the same idea, but a carry is generated whenever a column total exceeds 1. Since a hardware circuit can only handle a fixed number of bits at a time, a multi-bit adder must add each bit position and pass carries between positions.

2.3 Combinational Circuits

A combinational circuit is a digital circuit whose outputs depend only on the present values of its inputs. It has no memory and no clock. Half Adders, Full Adders, multiplexers, decoders and encoders are examples. The 4-bit Ripple Carry Adder is a combinational circuit: whenever A or B changes, the sum and

carry outputs settle to new values after a short propagation delay, and they do not depend on any earlier inputs.

2.4 Importance of Adders in Digital Systems

Adders are among the most heavily used building blocks in digital hardware. Subtraction, multiplication, division, address calculation, counters, comparison and many signal processing operations are all carried out using adders or circuits derived from them. The speed and size of the adder strongly influence the speed and size of a processor's arithmetic unit. A clear understanding of the simple Ripple Carry Adder is therefore the starting point for understanding faster and more advanced adder designs.

---

3. Objectives

The main objectives of this project are:

1. To understand binary addition and the logic behind carry generation and propagation.
2. To design a Half Adder and a Full Adder using basic logic gates (XOR, AND and OR).
3. To create the Full Adder as a reusable subcircuit in Logisim Evolution.
4. To design a 4-bit Ripple Carry Adder by cascading four Full Adders, with the initial carry input Cin = 0.
5. To add two 4-bit binary numbers and obtain a 4-bit sum and a final carry output.
6. To simulate the circuit for several input combinations and compare the observed outputs with the expected results.
7. To understand carry propagation delay and the limitation it places on the speed of the adder.
8. To gain practical experience in hierarchical (modular) digital circuit design.

---

4. Software and Tools Required

4.1 Logisim Evolution

Logisim Evolution is a free, open-source educational tool for designing and simulating digital logic circuits. It is an actively maintained continuation of the original Logisim by Carl Burch. The circuit is drawn on a canvas by placing components such as logic gates, input pins, output pins, splitters, constants and subcircuits, and then joining them with wires. The simulator evaluates the circuit continuously, so the effect of changing an input is seen immediately. The project file used here was saved by Logisim Evolution version 3.9.0, with the extension `.circ`.

4.2 Role in Designing and Testing Circuits

- **Graphical design:** gates and wires are placed visually, which makes the circuit easy to understand and debug.
- **Live simulation:** input pins can be toggled, or multi-bit values changed, and the outputs update at once.

- **Hierarchical design:** a circuit such as the Full Adder can be saved as a subcircuit and reused several times inside another circuit, in the same way that a function is reused in a program.
- **Wire colour feedback:** wires change colour to show logic levels. Bright green indicates logic 1, dark green indicates logic 0, and other colours signal errors or unconnected signals.
- **Multi-bit buses and splitters:** a 4-bit value can be carried on a single bus and separated into individual bits using a splitter.
- **Documentation:** circuits can be labelled, and screenshots or images can be exported for reports.

4.3 Requirements Summary

Item	Details
Software	Logisim Evolution (project file created with version 3.9.0)
Runtime	Java Runtime Environment (as required by the installed version)
Hardware	Any computer running Windows, macOS or Linux
Components used	Input pins, output pins, XOR gates, AND gates, OR gate, constant (logic 0), splitters, subcircuit
Project file	`RippleCarryAdder4Bit.circ`

---

5. Theoretical Background

5.1 Binary Number System

The binary number system has base 2 and uses only the digits 0 and 1. Each digit is called a bit. As in decimal numbers, the value of a bit depends on its position, but the weights are powers of 2 instead of powers of 10. For a 4-bit number b3 b2 b1 b0:

Value = $b3 \times 2^3 + b2 \times 2^2 + b1 \times 2^1 + b0 \times 2^0 = 8 \cdot b3 + 4 \cdot b2 + 2 \cdot b1 + b0$

For example, $1011_2 = 8 + 0 + 2 + 1 = 11_{10}$. The rightmost bit b0 is the Least Significant Bit (LSB) and the leftmost bit b3 is the Most Significant Bit (MSB). A 4-bit unsigned number can represent values from 0 to 15.

5.2 Binary Addition

Binary addition of two single bits follows four rules:

Operation	Result	Explanation
0 + 0	0	Sum 0, carry 0
0 + 1	1	Sum 1, carry 0
1 + 0	1	Sum 1, carry 0

Operation	Result	Explanation
1 + 1	10	Sum 0, carry 1

When a carry from the previous column is also present, three bits must be added in that column. The largest possible total is $1 + 1 + 1 = 11_2$, which gives sum 1 and carry 1.

Worked example (5 + 3):

```
Carry:   1 1 1 0   (carry into bit 3, 2, 1, 0)
A    =     0 1 0 1   (5)
B    =     0 0 1 1   (3)
---------------------
Sum  =     1 0 0 0   (8)
```

Bit 0: 1 + 1 = 10, so sum 0 and carry 1. Bit 1: 0 + 1 + 1 (carry) = 10, so sum 0 and carry 1. Bit 2: 1 + 0 + 1 (carry) = 10, so sum 0 and carry 1. Bit 3: 0 + 0 + 1 (carry) = 1, so sum 1 and carry 0. The result is $01000_2 = 8_{10}$.

When two 4-bit numbers are added, the result can need 5 bits. The fifth bit is the final carry (Cout), so the complete answer is written as Cout S3 S2 S1 S0.

5.3 Half Adder

A Half Adder adds two single bits, A and B, and produces a Sum bit and a Carry bit. It is called “half” because it cannot accept a carry coming from a previous stage.

5.4 Full Adder

A Full Adder adds three single bits: A, B and a carry input Cin. It produces a Sum bit and a carry output Cout. Because it accepts a carry input, Full Adders can be chained to add multi-bit numbers.

5.5 Ripple Carry Adder

A Ripple Carry Adder adds two n-bit numbers using n Full Adders connected in cascade. The Cout of stage i is connected to the Cin of stage i + 1. For an n-bit adder, n Full Adders are needed, and the output has n sum bits plus one final carry.

5.6 Carry Propagation

In a Ripple Carry Adder, the Full Adder at bit i cannot produce its correct Sum and Cout until the carry from bit i − 1 has arrived. A carry generated in the least significant stage may therefore have to pass through every higher stage, one after another. This sequential movement of the carry is called carry propagation. If each Full Adder introduces a delay of t_FA for the carry, the worst-case delay for an n-bit adder is approximately:

$t_{total} \approx n \times t_{FA}$

Thus the delay increases in direct proportion to the number of bits.

5.7 Sum and Carry Outputs

- The **Sum output** at each bit position is the result of adding the three input bits in that position and keeping only the least significant bit of the answer. It is 1 when an odd number of the inputs are 1.
- The **Carry output** is 1 when at least two of the three inputs are 1, because the total is then 2 or more and cannot fit in a single bit.
- In the 4-bit adder, S0 to S3 form the sum, and the carry out of the last stage is the final carry Cout. If Cout = 1, the true result is larger than 15 and needs a fifth bit.

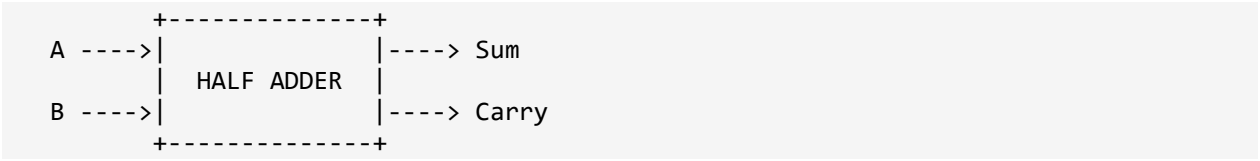
---

6. Half Adder Design

6.1 Definition

A Half Adder is a combinational circuit that adds two single-bit binary numbers, A and B, and produces two outputs: Sum and Carry. It has two inputs and two outputs and contains no carry input.

6.2 Block Diagram



6.3 Boolean Expressions

Sum = $A \oplus B$

Carry = $A \cdot B$

Here $\oplus$ denotes the XOR operation and $\cdot$ denotes the AND operation.

6.4 Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

6.5 Working Principle

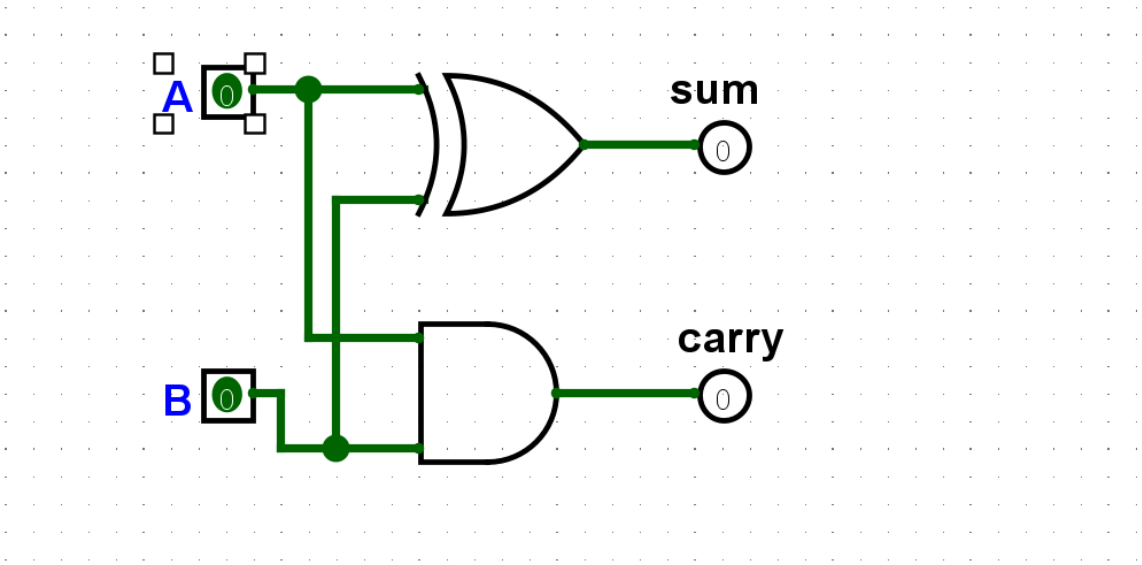
The XOR gate gives output 1 only when its two inputs are different. This matches the rule for the sum bit: 0 + 1 and 1 + 0 give sum 1, while 0 + 0 and 1 + 1 give sum 0 (the result 10 has a least significant bit of 0). The AND gate gives output 1 only when both inputs are 1. This matches the carry rule, because a

carry is generated only in the case 1 + 1. The circuit therefore needs only one XOR gate and one AND gate, and both gates receive the same two inputs A and B.

6.6 Role in Digital Arithmetic Circuits

The Half Adder is the simplest arithmetic circuit and is used to add the least significant bits of two numbers when no earlier carry exists. It is also the building block of the Full Adder: a Full Adder can be viewed as two Half Adders joined by an OR gate. Half Adders are also used in incrementers and in some counter designs.

Figure 1 shows the Half Adder built in Logisim Evolution. The XOR gate produces the Sum output and the AND gate produces the Carry output. Both input pins A and B are shown at logic 0 in this screenshot, so both outputs are also 0, which agrees with the first row of the truth table.



---

7. Full Adder Design

7.1 Definition

A Full Adder is a combinational circuit that adds three single-bit inputs, A, B and the carry input Cin, and produces a Sum bit and a carry output Cout. Because it can accept a carry from a previous stage, it is the basic cell for multi-bit adders.

7.2 Inputs and Outputs

Signal	Direction	Description
A	Input	First operand bit
B	Input	Second operand bit

Signal	Direction	Description
Cin	Input	Carry coming from the previous (less significant) stage
Sum	Output	Sum bit of the current stage
Cout	Output	Carry passed to the next (more significant) stage

7.3 Boolean Expressions

Sum = $A \oplus B \oplus Cin$

Cout = $(A \cdot B) + (Cin \cdot (A \oplus B))$

It is convenient to define an intermediate signal $X = A \oplus B$. The expressions then become:

Sum = $X \oplus Cin$

Cout = $(A \cdot B) + (X \cdot Cin)$

The term $A \cdot B$ is the carry **generated** by the current stage itself (both bits are 1). The term $X \cdot Cin$ is the incoming carry **propagated** through the stage (the two bits differ and a carry arrives). This is the same idea used later in Carry Look-Ahead Adders.

7.4 Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

7.5 Circuit Implementation Using Logic Gates

The Full Adder is implemented with five gates: two XOR gates, two AND gates and one OR gate. The labels below match those used on the Logisim schematic in Figure 2.

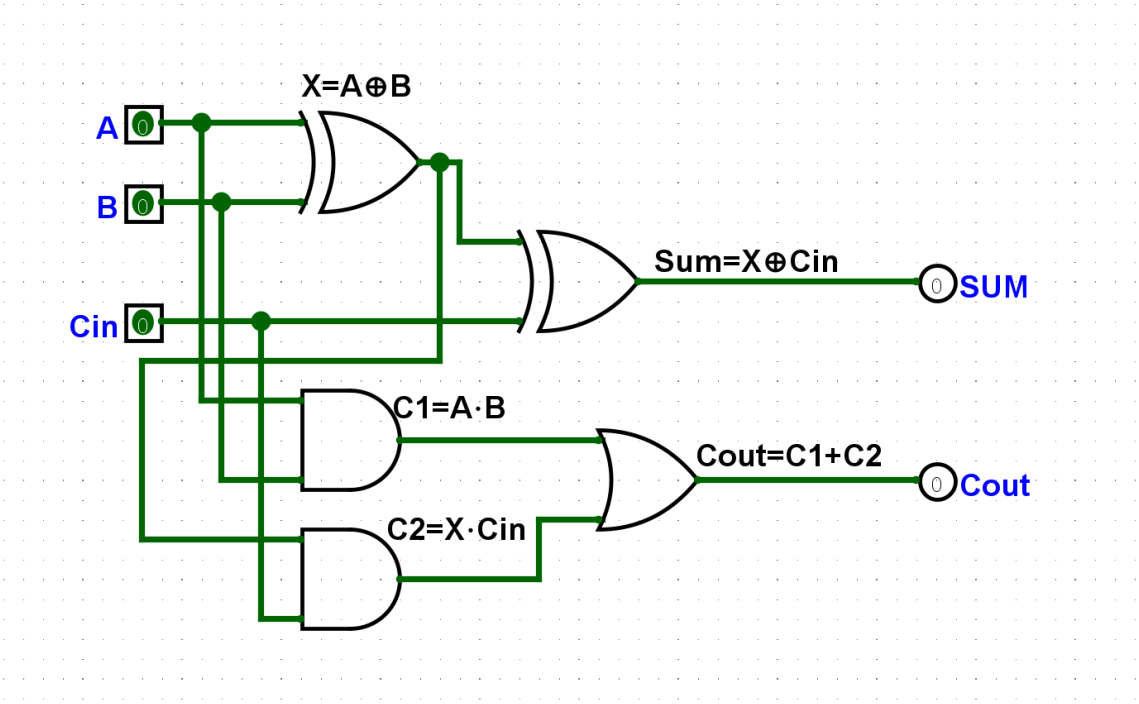
Gate	Inputs	Output	Purpose
XOR gate 1	A, B	$X = A \oplus B$	Finds whether the two operand bits differ

Gate	Inputs	Output	Purpose
XOR gate 2	X, Cin	Sum = X ⊕ Cin	Produces the final sum bit
AND gate 1	A, B	C1 = A · B	Carry generated by this stage
AND gate 2	X, Cin	C2 = X · Cin	Incoming carry propagated by this stage
OR gate	C1, C2	Cout = C1 + C2	Combines the two carry conditions

7.6 Explanation of Each Gate Connection

1. **XOR gate 1:** The input pins A and B are both connected to this gate. Its output X is 1 when A and B differ. X is used in two places: it goes to XOR gate 2 and to AND gate 2.
2. **XOR gate 2:** One input is X and the other is Cin. Its output is the Sum pin. In effect, this gate adds Cin to the partial sum of A and B.
3. **AND gate 1:** A and B are connected directly. Its output C1 is 1 only when both operand bits are 1, which means a carry is produced irrespective of Cin.
4. **AND gate 2:** X and Cin are connected. Its output C2 is 1 only when exactly one of A and B is 1 and a carry-in is present, so the carry-in must be passed on.
5. **OR gate:** C1 and C2 are never both 1 at the same time (C1 needs A = B = 1, while C2 needs A ≠ B), so the OR gate simply selects whichever carry condition is true. Its output is the Cout pin.

Figure 2 shows the Full Adder circuit as implemented in Logisim Evolution, with the intermediate signals X, C1 and C2 labelled on the schematic.



7.7 Difference Between a Half Adder and a Full Adder

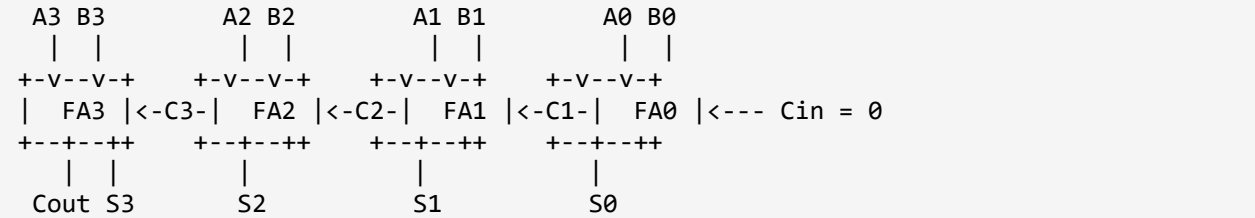
Feature	Half Adder	Full Adder
Number of inputs	2 (A, B)	3 (A, B, Cin)
Outputs	Sum, Carry	Sum, Cout
Carry input	Not available	Available
Sum equation	$A \oplus B$	$A \oplus B \oplus Cin$
Carry equation	$A \cdot B$	$A \cdot B + Cin \cdot (A \oplus B)$
Gates needed	1 XOR, 1 AND	2 XOR, 2 AND, 1 OR
Can be cascaded	No	Yes
Typical use	Adding the LSBs, incrementers	Every bit position of a multi-bit adder

---

8. 4-Bit Ripple Carry Adder Architecture

8.1 Cascade Connection of Four Full Adders

To add two 4-bit numbers, four Full Adders (FA0, FA1, FA2 and FA3) are used, one for each bit position. FA0 handles the least significant bits and FA3 handles the most significant bits. The carry output of each Full Adder is connected to the carry input of the next one.



8.2 Inputs and Outputs

Signal	Description
A3, A2, A1, A0	The four bits of the first operand A. A3 is the MSB and A0 is the LSB.
B3, B2, B1, B0	The four bits of the second operand B. B3 is the MSB and B0 is the LSB.
Cin = 0	Initial carry input to FA0. It is fixed at logic 0 because there is no earlier stage.
S3, S2, S1, S0	The four sum output bits.
Cout	Final carry output from FA3. It is the fifth bit of the result.

The complete result is the 5-bit number Cout S3 S2 S1 S0.

8.3 Stage-by-Stage Equations

Let C0 = Cin = 0. For each bit position i = 0, 1, 2, 3:

$S_i = A_i \oplus B_i \oplus C_i$

$C_{i+1} = (A_i \cdot B_i) + (C_i \cdot (A_i \oplus B_i))$

and the final carry is Cout = C4.

8.4 Carry Propagation from One Full Adder to the Next

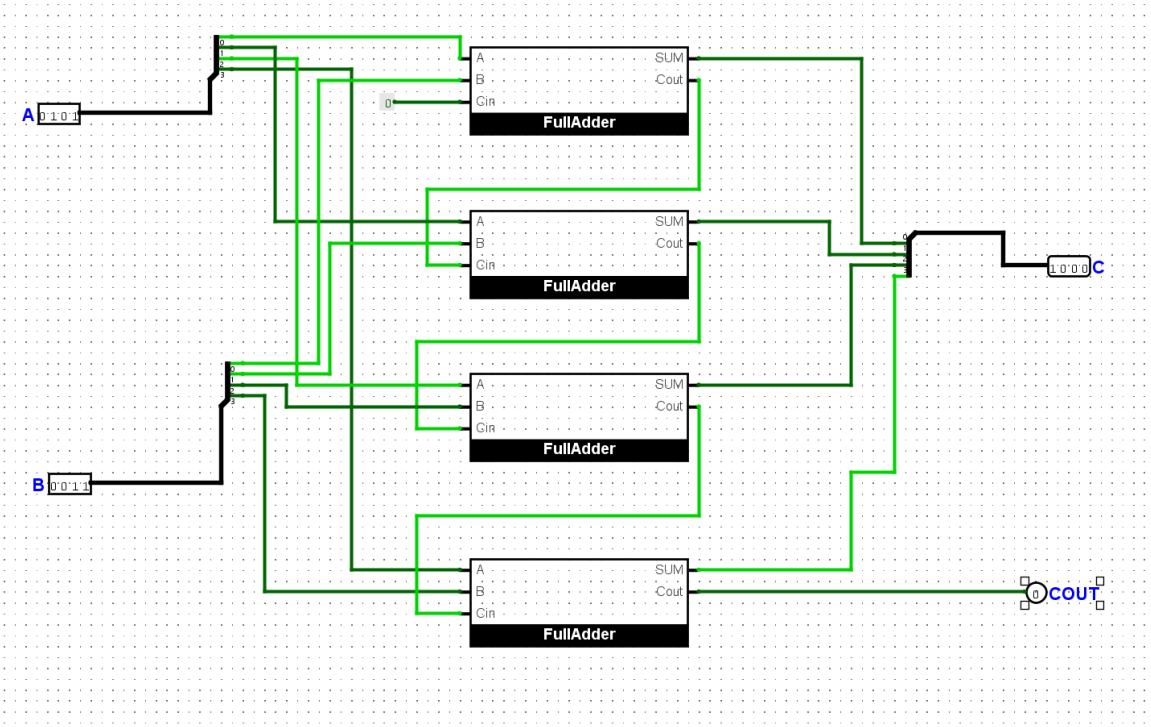
- FA0 adds A0, B0 and Cin = 0, and produces S0 and the carry C1.
- FA1 adds A1, B1 and C1, and produces S1 and the carry C2.
- FA2 adds A2, B2 and C2, and produces S2 and the carry C3.
- FA3 adds A3, B3 and C3, and produces S3 and the final carry Cout.

A stage can only finish its calculation correctly after the carry from the stage before it is available.

8.5 Why It Is Called a Ripple Carry Adder

The carry signal moves through the adder from the least significant stage to the most significant stage in sequence, like a ripple moving across water. For example, when 1111 is added to 0001, the carry created in FA0 changes the output of FA1, which creates a carry that changes FA2, and so on up to FA3. The final result is correct only after the carry has rippled through all four stages. This also explains why the delay of the adder increases with the number of bits.

Figure 3 shows the complete 4-bit Ripple Carry Adder built in Logisim Evolution. Four FullAdder subcircuits are stacked vertically, the carry output of each is wired to the carry input of the one below it, and the first carry input is connected to a constant 0. The inputs A and B are 4-bit input pins whose bits are separated by splitters, and the four sum bits are combined by a splitter into the 4-bit output C. The final carry is shown on the output pin COUT.



---

9. Circuit Design in Logisim Evolution

This chapter gives step-by-step instructions that a beginner can follow to build the circuit. Menu names may differ slightly between versions of Logisim Evolution.

9.1 Creating a New Project

1. Install a suitable Java runtime, then download and start Logisim Evolution from its official repository.
2. A new project opens automatically with a circuit named **main**. If it does not, choose **File → New**.
3. Choose **File → Save As** and save the project immediately as `RippleCarryAdder4Bit.circ`.
4. The left panel shows the component library (Wiring, Gates, Plexers, Arithmetic and others) and the project's circuits. The large central area is the drawing canvas. Use the **Edit tool** (arrow) to select and move components, and the **Poke tool** (hand) to change input values during simulation.

9.2 Creating the Full Adder Subcircuit

1. Choose **Project → Add Circuit**, type the name **FullAdder** and press OK.
2. The canvas now shows the empty FullAdder circuit. All the following steps in 9.3 to 9.6 are performed in this circuit.

9.3 Adding Input and Output Pins

1. Open the **Wiring** folder in the library panel and select **Pin**. Click on the canvas three times on the left side to place three input pins.

2. Select each pin with the Edit tool and use the Properties panel to set its **Label** to **A**, **B** and **Cin** in turn. Leave the data bits at 1.
3. Select **Pin** again, place two pins on the right side of the canvas, and in the Properties panel set **Output?** to **Yes**. Label them **SUM** and **Cout**.
4. Keep the inputs on the left and the outputs on the right. Logisim uses these positions to decide where the pins appear on the subcircuit symbol.

9.4 Placing the XOR, AND and OR Gates

1. Open the **Gates** folder. Select **XOR Gate** and click twice on the canvas to place two XOR gates.
2. Select **AND Gate** and place two AND gates.
3. Select **OR Gate** and place one OR gate.
4. Arrange them as in Figure 2: the first XOR near the inputs, the second XOR to its right, the two AND gates below, and the OR gate at the right of the AND gates.
5. Make sure each gate has 2 inputs and 1 data bit in its Properties panel (the default).

9.5 Connecting the Wires

Select the Edit tool, then click on an output or input terminal and drag to draw a wire. A wire bends at right angles and a junction dot appears when more than two wire ends meet.

1. Connect **A** and **B** to the two inputs of **XOR gate 1**. This output is $X = A \oplus B$.
2. Connect **X** to one input of **XOR gate 2** and connect **Cin** to its other input. Connect the output of XOR gate 2 to the **SUM** pin.
3. Connect **A** and **B** to the two inputs of **AND gate 1**. This output is $C1 = A \cdot B$.
4. Connect **X** and **Cin** to the two inputs of **AND gate 2**. This output is $C2 = X \cdot Cin$.
5. Connect **C1** and **C2** to the two inputs of the **OR gate**, and connect the OR output to the **Cout** pin.
6. Check that no wire is red or blue (error or unknown). Optionally add text labels such as X, C1 and C2 with the **Text** tool, as done in Figure 2.

9.6 Testing the Full Adder

1. Choose the **Poke tool**.
2. Click on the input pins A, B and Cin to toggle each between 0 and 1.
3. For each of the eight combinations, compare SUM and Cout with the Full Adder truth table in Section 7.4.
4. Record the observations in the table provided in Section 11.2. Do not move on until all eight rows match.

9.7 Creating the Main 4-Bit Ripple Carry Adder Circuit

1. In the left panel, double-click the circuit **main** to open it.
2. Place a 4-bit input pin for A: select **Pin**, click on the canvas, and in the Properties panel set **Data Bits = 4** and **Label = A**. Do the same for **B**.
3. Place an output pin, set **Output? = Yes**, **Data Bits = 4** and label it **C** (the 4-bit sum). Place another output pin with 1 data bit and label it **COUT**.

4. Open the Wiring folder and select **Splitter**. Set **Fan Out = 4** and **Bit Width In = 4**. Place one splitter next to input A and another next to input B. Each splitter separates the 4-bit bus into bits 0, 1, 2 and 3. Place a third splitter near output C to combine four single bits into one 4-bit bus.
5. Select **Constant** from the Wiring folder, set its value to **0** and place it near the first Full Adder. This will be Cin = 0.

9.8 Placing Four Full Adder Components

1. In the left panel, click the **FullAdder** circuit once to select it as the current tool.
2. Click four times on the canvas to place four instances in a vertical column. Each instance appears as a rectangular box with pins A, B, Cin, SUM and Cout.
3. Order them from top to bottom as FA0, FA1, FA2 and FA3, so that FA0 handles the least significant bit.

9.9 Connecting the Carry Chain

1. Connect the **constant 0** to the **Cin** pin of FA0.
2. Connect the **Cout** of FA0 to the **Cin** of FA1.
3. Connect the **Cout** of FA1 to the **Cin** of FA2.
4. Connect the **Cout** of FA2 to the **Cin** of FA3.

The carry chain is now complete. The chain runs in one direction only, from the least significant stage to the most significant stage.

9.10 Connecting the Input and Output Pins

1. Connect splitter bit 0 of **A** to the A pin of FA0, bit 1 to FA1, bit 2 to FA2 and bit 3 to FA3.
2. Connect splitter bit 0 of **B** to the B pin of FA0, bit 1 to FA1, bit 2 to FA2 and bit 3 to FA3.
3. Connect the **SUM** pin of FA0, FA1, FA2 and FA3 to bits 0, 1, 2 and 3 of the output splitter, and connect the output splitter to the 4-bit output pin **C**.
4. Connect the **Cout** of FA3 to the output pin **COUT**.
5. Take care that bit 0 is always matched to FA0 (the LSB) for A, B and the sum. A wrong bit order is the most common cause of wrong answers.

9.11 Saving the Circuit File

1. Choose **File → Save** (Ctrl + S) and keep the file name `RippleCarryAdder4Bit.circ`.
2. The single file contains both circuits, main and FullAdder.
3. Keep a backup copy and also export screenshots of the circuits using **File → Export Image** for use in the report.

---

10. Truth Tables

10.1 Half Adder

A	B	Sum	Carry

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

10.2 Full Adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

In every row, the number of 1s among A, B and Cin equals the value of the two-bit number Cout Sum. For example, in the row 1, 1, 0 there are two 1s and the output Cout Sum is 10_2, which is 2. This is a quick way to check the table.

---

11. Testing and Verification

11.1 Test Procedure

1. Open `RippleCarryAdder4Bit.circ` in Logisim Evolution and open the **main** circuit.
2. Select the **Poke tool**.
3. Click on the 4-bit input pin A. The value of each bit can be changed by clicking on the individual bit boxes. Set the required binary value.
4. Do the same for input pin B.
5. Observe the 4-bit output pin C (the sum bits S3 S2 S1 S0) and the single-bit output COUT. The wire colour also shows the logic levels: bright green is 1 and dark green is 0.
6. Write the combined result as Cout followed by S3 S2 S1 S0 and compare it with the expected value.
7. Take a screenshot of each test for inclusion in Chapter 12.

11.2 Full Adder Verification Table (to be completed by the student)

Test the FullAdder subcircuit for all eight input combinations and fill in the observed columns.

A	B	Cin	Expected Sum	Expected Cout	Observed Sum	Observed Cout
0	0	0	0	0	[ ]	[ ]
0	0	1	1	0	[ ]	[ ]
0	1	0	1	0	[ ]	[ ]
0	1	1	0	1	[ ]	[ ]
1	0	0	1	0	[ ]	[ ]
1	0	1	0	1	[ ]	[ ]
1	1	0	0	1	[ ]	[ ]
1	1	1	1	1	[ ]	[ ]

11.3 Test Cases for the 4-Bit Ripple Carry Adder

The expected results below were calculated by hand using binary addition. The result is written as Cout S3 S2 S1 S0.

Test	A	B	Decimal	Expected result (Cout S3 S2 S1 S0)	Observed result	Status
1	0101	0011	5 + 3 = 8	01000	Cout = 0, C = 1000 (Figure 3)	Verified from screenshot
2	0111	0010	7 + 2 = 9	01001	[to be entered]	Pending
3	1111	0001	15 + 1 = 16	10000	[to be entered]	Pending
4	1010	0101	10 + 5 = 15	01111	[to be entered]	Pending
5	1111	1111	15 + 15 = 30	11110	[to be entered]	Pending

Note on verification status. Only Test 1 is supported by a simulation screenshot in this report (Figure 3, where A = 0101, B = 0011, C = 1000 and COUT = 0). Tests 2 to 5 are expected values only. They must be run in Logisim Evolution, and their screenshots inserted in Chapter 12, before the status is changed to "Verified". The Half Adder and Full Adder screenshots (Figures 1 and 2) show only the all-zero input state, so they confirm correct wiring of the first truth-table row but do not by themselves verify the complete truth tables.

11.4 Hand Calculation of the Test Cases

- **Test 1:** 0101 + 0011. Bit 0: 1 + 1 = 0, carry 1. Bit 1: 0 + 1 + 1 = 0, carry 1. Bit 2: 1 + 0 + 1 = 0, carry 1. Bit 3: 0 + 0 + 1 = 1, carry 0. Result 01000.

- **Test 2:** 0111 + 0010. Bit 0: 1 + 0 = 1. Bit 1: 1 + 1 = 0, carry 1. Bit 2: 1 + 0 + 1 = 0, carry 1. Bit 3: 0 + 0 + 1 = 1, carry 0. Result 01001.
- **Test 3:** 1111 + 0001. Bit 0: 1 + 1 = 0, carry 1. Bits 1, 2 and 3: 1 + 0 + 1 = 0, carry 1 each time. Result 10000. This test makes the carry ripple through all four stages.
- **Test 4:** 1010 + 0101. Every bit position has exactly one 1 and no carry is generated, so the sum is 1111 and Cout = 0. Result 01111.
- **Test 5:** 1111 + 1111. Bit 0: 1 + 1 = 0, carry 1. Bits 1, 2 and 3: 1 + 1 + 1 = 1, carry 1. Result 11110.

11.5 Interpreting the Final Carry and Sum Outputs

- The four sum outputs S3 S2 S1 S0 form the lower four bits of the result, and the final carry Cout is the fifth (most significant) bit. The full answer is Cout S3 S2 S1 S0.
- If Cout = 0, the result fits in four bits and the sum bits give the answer directly (for example, 01000 means 8).
- If Cout = 1, the true answer is 16 or more. The value is 16 + (S3 S2 S1 S0). For example, 11110 means 16 + 14 = 30.
- For unsigned numbers, Cout = 1 also indicates that the 4-bit sum alone is not enough to represent the answer (overflow of the 4-bit range).

---

12. Circuit Diagrams and Screenshots

Figures 1, 2 and 3 in Chapters 6 to 8 are the student's actual Logisim Evolution screenshots of the Half Adder, the Full Adder and the complete 4-bit Ripple Carry Adder. The placeholders below are for the remaining screenshots. **Insert only genuine screenshots taken from the simulation. Do not use images from other sources.**

Figure	Content	Status
Figure 1	Half Adder circuit	Inserted in Section 6.6
Figure 2	Full Adder circuit	Inserted in Section 7.6
Figure 3	Complete 4-bit Ripple Carry Adder (Test 1 state)	Inserted in Section 8.5
Figure 4	Half Adder simulation for all four input combinations	[INSERT SCREENSHOT]
Figure 5	Full Adder simulation for all eight input combinations	[INSERT SCREENSHOT]
Figure 6	Simulation of Test 2 (A = 0111, B = 0010)	[INSERT SCREENSHOT]
Figure 7	Simulation of Test 3 (A = 1111, B = 0001)	[INSERT SCREENSHOT]
Figure 8	Simulation of Test 4 (A = 1010, B = 0101)	[INSERT SCREENSHOT]
Figure 9	Simulation of Test 5 (A = 1111, B = 1111)	[INSERT SCREENSHOT]
Figure 10	Final output screenshot (summary of results)	[INSERT SCREENSHOT]

[PLACEHOLDER: Figure 4 – Half Adder simulation screenshots]

[PLACEHOLDER: Figure 5 – Full Adder simulation screenshots]

[PLACEHOLDER: Figure 6 – Test 2 simulation screenshot]

[PLACEHOLDER: Figure 7 – Test 3 simulation screenshot]

[PLACEHOLDER: Figure 8 – Test 4 simulation screenshot]

[PLACEHOLDER: Figure 9 – Test 5 simulation screenshot]

[PLACEHOLDER: Figure 10 – Final output screenshot]

---

13. Results and Discussion

13.1 Expected Behaviour

The 4-bit Ripple Carry Adder is expected to add the two 4-bit unsigned inputs A and B (with Cin = 0) and to produce the 5-bit result Cout S3 S2 S1 S0, which equals A + B. Each Full Adder follows the truth table in Section 10.2, and each carry output feeds the next stage. As the inputs change, the outputs settle after the carry has rippled through the chain.

13.2 Experimentally Verified Results

The simulation screenshot of the complete circuit (Figure 3) shows the inputs A = 0101 and B = 0011 and the outputs C = 1000 and COUT = 0. This corresponds to the result 01000, which equals the decimal value 8 and matches the expected value for 5 + 3. This is the only test case verified by a screenshot in this report.

The screenshots of the Half Adder and Full Adder (Figures 1 and 2) were taken with all inputs at 0. They show Sum = 0, Carry = 0 and Cout = 0, which is consistent with the first row of each truth table.

13.3 Expected but Not Yet Verified Results

Tests 2 to 5 (7 + 2, 15 + 1, 10 + 5 and 15 + 15) have expected results that were obtained by hand calculation. They should be marked as verified in Section 11.3 only after the student has run them in Logisim Evolution and attached the screenshots.

13.4 Discussion

- The outputs show binary addition: each sum bit is the result of adding three bits in one column, and each carry is passed to the next column, exactly as in manual column addition.
- Test 1 demonstrates carry propagation through three stages, since carries are produced in FA0, FA1 and FA2.
- Test 3 (15 + 1) is the worst-case test, because the carry created in the first stage must travel through all four stages to the final carry output.
- Test 5 (15 + 15) produces the largest possible sum for 4-bit inputs with Cin = 0, which is 30, and it uses the fifth bit (Cout).

- In Logisim Evolution the results appear to change immediately, because the simulator does not model real gate delays by default. In real hardware, the output would take a short time to settle, and this settling time is the carry propagation delay discussed in Chapter 15.

---

14. Applications

- **Arithmetic Logic Units (ALUs):** The adder is the core of the ALU, which carries out addition, subtraction (by adding the two's complement), comparison and address calculations.
- **Microprocessors:** Processors use adders in the ALU, to increment the program counter, to compute memory addresses and in floating-point and multiplication units.
- **Digital calculators:** Calculators perform arithmetic on binary or BCD values using adder circuits.
- **Embedded systems:** Microcontrollers use small adders for timers, counters, pointer arithmetic and control calculations, where simple, low-area designs are valuable.
- **Digital signal processing:** Filters, FFT computations and multiply-accumulate operations all need large numbers of additions.
- **General-purpose arithmetic circuits:** Adders are used in counters, subtractors, comparators, multipliers and address generators.

---

15. Advantages and Limitations

15.1 Advantages

- **Simplicity:** The architecture is just a chain of identical Full Adders and is easy to understand and teach.
- **Modularity:** One FullAdder design is reused, so the circuit can be extended to any word length by adding more stages.
- **Ease of implementation:** Few gates and regular wiring make it simple to design, simulate and debug.
- **Small area:** It uses fewer gates than faster adders such as the Carry Look-Ahead Adder, which means lower cost and power.

15.2 Limitations

- **Carry propagation delay:** Each stage must wait for the carry from the previous stage, so the worst-case delay is roughly n × t_FA for an n-bit adder.
- **Poor scalability for wide words:** For 32-bit or 64-bit adders, the delay becomes too long for high-speed processors.
- **Comparison with faster adders:** Carry Look-Ahead, Carry Select, Carry Skip and prefix adders (such as Kogge–Stone) calculate carries faster, by using more hardware.
- **Unsigned result only in this design:** The circuit as built does not include overflow detection for signed numbers or subtraction.

Feature	Ripple Carry Adder	Carry Look-Ahead Adder

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Hardware complexity	Low	Higher
Delay growth	Proportional to number of bits	Much slower growth
Design effort	Very simple	More complex
Typical use	Small adders, teaching, low-area designs	High-speed processors

---

16. Learning Outcomes

Through this project the following concepts and skills were gained:

1. A clear understanding of the binary number system and binary addition with carry.
2. Ability to derive and use Boolean expressions for the Half Adder and Full Adder, and to verify them with truth tables.
3. Understanding of the roles of XOR, AND and OR gates in arithmetic circuits.
4. Understanding of generate and propagate terms, which are the basis of faster adders.
5. Skills in hierarchical design: creating a subcircuit and reusing it several times.
6. Practical use of Logisim Evolution: placing components, using pins, splitters, constants and wires, and testing with the Poke tool.
7. Understanding of carry propagation delay and of the speed and area trade-off.
8. Habits of testing and documentation: preparing test cases, comparing expected and observed results, and recording only verified results.

---

17. Conclusion

This project designed and simulated a 4-bit Ripple Carry Adder in Logisim Evolution. A Half Adder was built from an XOR gate and an AND gate, and a Full Adder was built from two XOR gates, two AND gates and one OR gate, using Sum = A $\oplus$ B $\oplus$ Cin and Cout = A·B + Cin·(A $\oplus$ B). The Full Adder was saved as a subcircuit, and four copies were cascaded with Cin = 0 at the first stage, with 4-bit inputs A and B, a 4-bit sum output C and a final carry output COUT.

The simulation screenshot for A = 0101 and B = 0011 shows the output 01000, which agrees with the expected value of 5 + 3 = 8. The remaining test cases have been defined with expected values and are to be verified in the simulator. The project shows how a complex arithmetic circuit can be built from small, simple, reusable blocks, and how carry propagation limits the speed of the Ripple Carry Adder. It forms a good base for the study of faster adders and ALU design.

---

18. Future Scope

- **8-bit and 16-bit adders:** Extend the design by cascading more FullAdder stages, or by combining two 4-bit adders, and study how the delay grows.
- **Carry Look-Ahead Adder:** Use the generate (Gi = Ai·Bi) and propagate (Pi = Ai ⊕ Bi) signals to compute the carries in parallel, and compare delay and gate count with the RCA.
- **Subtractor circuits:** Add XOR gates on input B and set Cin = 1 to build an adder/subtractor using two's complement.
- **Arithmetic Logic Unit design:** Combine the adder with AND, OR, XOR and shifting units and a multiplexer for operation selection to build a small ALU.
- **FPGA implementation:** Describe the circuit in Verilog or VHDL, synthesize it for an FPGA board, and measure timing and resource use.
- **Overflow and flag outputs:** Add overflow, zero and negative flags for signed arithmetic.

---

19. References

1. M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, Pearson.
2. Thomas L. Floyd, *Digital Fundamentals*, Pearson.
3. Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill.
4. David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture*, Morgan Kaufmann.
5. R. P. Jain, *Modern Digital Electronics*, McGraw-Hill Education.
6. Logisim-evolution official repository and documentation: https://github.com/logisim-evolution/logisim-evolution

7. Carl Burch, *Logisim* (the original software on which Logisim Evolution is based): http://www.cburch.com/logisim/